Ground Control Station

Telemetry Reception, Display and Logging Software

CanSat / Rocketry Avionics Programme

Technical Reference and Field Operations Manual

Document	Ground Station Software Documentation
Revision	1.0
Date	October 26, 2026
Subsystem	Ground Segment / Telemetry
Flight unit	Teensy 4.1 + Digi XBee 3 PRO (XB3-24AST)
RF standard	IEEE 802.15.4, 2.4 GHz, point-to-point
Software	Python 3.10–3.12, PyQt5, pyqtgraph, pyserial

Scope. This document describes the ground segment software: the telemetry packet format it accepts, its internal threading architecture, the dashboard user interface, the verification testing performed against it, and the complete procedure for connecting it to the flight hardware over the real RF link. It is intended both as a maintenance reference for the student avionics team and as a design-review supporting document.

Contents

1 Overview and Purpose **4**
1.1 What this software is . . . 4
1.2 What this software is deliberately *not* . . . 4
1.3 Position in the avionics architecture . . . 4
1.4 RF link parameters . . . 5

2 Telemetry Packet Format **5**
2.1 Frame structure . . . 5
2.2 Field definitions . . . 6
2.3 XOR checksum . . . 6
2.4 Flight state machine . . . 7

3 Software Architecture **7**
3.1 Module layout . . . 7
3.1.1 `telemetry_packet.py` — packet model and parser . . . 7
3.1.2 `serial_worker.py` — ingestion thread . . . 9
3.1.3 `csv_logger.py` — disk logging thread . . . 10
3.1.4 `dashboard_ui.py` — main window . . . 10
3.1.5 `main.py` — entry point . . . 10
3.1.6 `packet_sim.py` — synthetic source . . . 11
3.2 Threading model . . . 11
3.2.1 Why serial I/O is not on the GUI thread . . . 11
3.2.2 Why disk I/O is not on the GUI thread . . . 11
3.2.3 Render decoupling — the reason the UI never freezes . . . 12
3.3 Ring buffer and frame synchronisation . . . 12
3.3.1 The problem . . . 12
3.3.2 The ring buffer . . . 12
3.3.3 The frame-sync rules . . . 13
3.3.4 Worked recovery example . . . 13
3.3.5 Three layers of defence . . . 14

4 Dashboard UI Reference **14**
4.1 Overall layout . . . 14
4.2 Connection bar . . . 14
4.3 Flight state banner and live telemetry . . . 14
4.4 Strip charts . . . 15
4.5 GPS / NavIC panel . . . 15
4.6 Link status panel . . . 17
4.7 Display settings and event log . . . 17

5 Testing and Validation Performed **17**
5.1 Approach . . . 17
5.2 Test matrix . . . 18
5.3 Fault injection with `packet_sim.py` . . . 19
5.4 Recommended pre-flight regression . . . 20

6 Connecting to Real Hardware **20**
6.1 Ground-side XBee setup . . . 20
6.1.1 Hardware required . . . 20
6.1.2 Wiring . . . 21
6.1.3 Configuration in XCTU . . . 21

6.1.4 API mode versus transparent mode . . . 21
6.2 Selecting the correct COM port . . . 22
6.2.1 Identifying the port . . . 22
6.2.2 Selecting it in the dashboard . . . 22
6.2.3 When several COM ports appear . . . 23
6.3 Baud rate matching and the serial budget . . . 23
6.4 First real link test . . . 24
6.5 After Teensy 4.1 integration . . . 25
6.5.1 What changes on the ground: nothing . . . 25
6.5.2 What to verify . . . 25
6.6 Field deployment . . . 26
6.6.1 Ground antenna . . . 26
6.6.2 Laptop power for a session over two hours . . . 26
6.6.3 RF troubleshooting: the stale-telemetry checklist . . . 27
6.7 CSV handoff for competition judging . . . 28
6.7.1 Where the files are . . . 28
6.7.2 Retention of the raw frame . . . 28
6.7.3 Pre-handoff verification . . . 28

7 Live Map and KML Export **29**
7.1 Live map . . . 29
7.1.1 Offline is the base case . . . 29
7.1.2 Threading . . . 30
7.1.3 Display . . . 30
7.1.4 Sources . . . 30
7.1.5 Building a pack . . . 31
7.2 KML export . . . 31

8 Known Limitations and Future Work **31**
8.1 Functional limitations . . . 32
8.2 Technical debt and platform notes . . . 32
8.3 Suggested priority order . . . 33

A Quick Reference **33**
A.1 Installation and launch . . . 33
A.2 Simulator . . . 33
A.3 Configuration constants . . . 34
A.4 Symptom index . . . 34

List of Figures

1 End-to-end telemetry chain. The GCS software covers the dashed region: everything from raw serial bytes to the archived flight log. 4
2 Three-thread model. Ownership is exclusive: exactly one thread may touch the serial port, exactly one may touch the log files, and exactly one may touch widgets. 11
3 Main dashboard during a simulated flight. Left column: flight-state banner, live telemetry tiles, GPS/NavIC panel with ground track, link status and display settings. Right: altitude, pressure, temperature, battery voltage, accelerometer and gyroscope strip charts. 15
4 GPS/NavIC panel: latitude, longitude, navigation altitude and navigation time readouts above the two-dimensional ground track. 16
5 Link status panel showing packet rate, last packet age, valid/total packet counts, corrupt packet count and the link health banner. 17

List of Tables

1 RF configuration per vehicle. 5
2 Telemetry field definitions, units and expected ranges. 6
3 Worked XOR checksum for the body `ABC`. 7
4 Flight state machine states and dashboard colour coding. 8
5 Source module responsibilities. 8
6 Connection bar controls. 14
7 Strip chart contents. 16
8 Link status fields. 17
9 Ground station verification test matrix. 18
10 Fault injection modes and the real-world failure each simulates. 20
11 XCTU parameters for the ground radio (CanSat values shown). 21
12 Serial throughput budget for a 127-byte frame (125-byte frame plus `CRLF`), at 10 bits per transmitted byte. 24
13 Stale telemetry troubleshooting sequence. 27
14 Output files. 28
15 Map sources. No Google Maps or Google tile server is used anywhere. 30
16 Tunable constants and where they live. 34
17 Common symptoms and their usual causes. 35

1 Overview and Purpose

1.1 What this software is

The Ground Control Station (GCS) is the ground-segment terminal for the vehicle's telemetry downlink. It is a Python 3 desktop application built on PyQt5 and pyqtgraph whose single responsibility is to *receive, validate, display and archive* the telemetry stream produced by the flight computer.

Concretely, the GCS:

(a) opens a serial port connected to the ground-side XBee radio and reads the incoming byte stream continuously;

(b) recovers packet boundaries from that stream, which is not frame-aligned and which will contain dropped, duplicated and corrupted bytes after any real RF link impairment;

(c) verifies the integrity of every recovered packet with an XOR checksum and rejects the ones that fail, counting them rather than displaying them;

(d) renders the validated telemetry as numeric readouts, six scrolling strip charts, a GPS ground track and a colour-coded flight-state banner;

(e) writes every validated packet to a CSV flight log on disk, together with a ground-station receive timestamp, so the flight can be analysed afterwards and the log submitted for competition scoring;

(f) archives every *rejected* frame, byte for byte, to a separate error log so that link problems can be diagnosed after the fact.

1.2 What this software is deliberately *not*

The GCS is a receive-only terminal. It has no command uplink: it cannot arm, disarm, calibrate or reconfigure the flight computer. This is a deliberate scope decision — a one-way link removes an entire class of failure modes (accidental command transmission, uplink jamming, ground software defects reaching the vehicle) from the flight-critical path. Any future uplink capability is discussed in Section 8.

1.3 Position in the avionics architecture

The telemetry chain runs from the flight computer to the operator's screen through six stages, shown in Figure 1. The GCS occupies the last two.

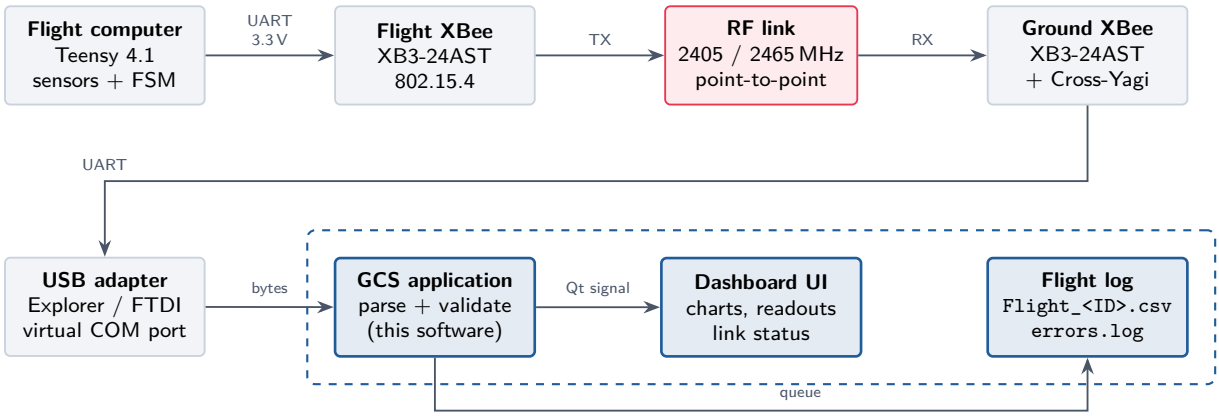


Figure 1: End-to-end telemetry chain. The GCS software covers the dashed region: everything from raw serial bytes to the archived flight log.

1.4 RF link parameters

Both vehicles use the Digi XBee 3 PRO (XB3-24AST) operating under IEEE 802.15.4 in point-to-point mode. Mesh/DigiMesh networking is *not* used: a single coordinator-to-endpoint association keeps latency deterministic and removes routing overhead from a link that carries a continuous 20 Hz stream.

In the 2.4 GHz 802.15.4 band, channel centre frequencies follow

$$f_c = 2405 + 5\,(k - 11)\ \text{MHz}, \qquad k = 11 \ldots 26, \tag{1}$$

so the two mission frequencies map onto the XBee `CH` parameter as shown in Table 1. The `CH` register is set in hexadecimal in XCTU.

Table 1: RF configuration per vehicle.

Vehicle	Frequency	802.15.4 channel (`CH`)	Allocation
CanSat	2465 MHz	23 (`0x17`)	IN-SPACe channel assignment
Rocket	2405 MHz	11 (`0x0B`)	IN-SPACe channel assignment

Both radios in a pair must match on three parameters

The flight unit and the ground unit must agree on `CH` (channel), `ID` (PAN ID) and `BD` (serial baud rate). A mismatch on `CH` or `ID` produces *complete silence* with no error message anywhere — the dashboard will simply show AWAITING TELEMETRY forever. This is by far the most common cause of a dead link on the bench, and the first thing to check in the field. See Section 6.

2 Telemetry Packet Format

2.1 Frame structure

Telemetry is transmitted as a single line of printable ASCII per sample:

```
$TEAM_ID,TIMESTAMP,PACKET_COUNT,ALTITUDE,PRESSURE,TEMP,VOLTAGE,NAV_TIME,
  LAT,LON,NAV_ALT,SATS,ACC_X,ACC_Y,ACC_Z,GYRO_X,GYRO_Y,GYRO_Z,FSM_STATE*CS
```

The structure follows NMEA-0183 convention:

- `$` — start delimiter, one byte (`0x24`). Marks the beginning of a frame.
- *body* — exactly 19 comma-separated fields. The body must not itself contain `$` or `*`.
- `*` — end delimiter, one byte (`0x2A`).
- `CS` — exactly two uppercase or lowercase hexadecimal digits holding the XOR checksum of the body.

A trailing `CR LF` is optional; the parser tolerates its presence or absence, and any bytes between the checksum and the next `$` are discarded as inter-frame junk.

A real frame, taken verbatim from the packet simulator, is:

```
$2025ASI001,0.05,2,320.04,975.39,25.90,8.20,012829.00,32.726600,
74.857001,317.81,4,-0.031,0.049,9.636,5.240,-1.444,1.506,0*43
```

This frame is 125 bytes long, or 127 bytes with `CR LF`. That number drives the link budget in Section 6.3 and it is worth remembering.

2.2 Field definitions

Table 2 defines all 19 fields. The *Strictness* column records how `telemetry_packet.py` treats a blank or unparseable value:

Mandatory
A blank or non-numeric value raises `PacketParseError` and the *entire packet is rejected*. These are the fields without which the packet has no meaning.

Optional
A blank or non-numeric value is substituted with `NaN` (or `0` for integer counts) and the packet is *still accepted*. This exists because a GNSS receiver legitimately transmits empty position fields before it achieves a fix; discarding those packets would blind the operator to altitude and flight state during the most critical part of the mission.

Table 2: Telemetry field definitions, units and expected ranges.

#	Field	Type	Unit	Strictness	Expected range / notes
1	`TEAM_ID`	string	—	Mandatory	Competition team identifier, e.g. `2025ASI001`. Must be non-empty. Determines the CSV file name.
2	`TIMESTAMP`	float	s	Mandatory	Mission time since flight-computer boot, 0–7200. Also accepts `HH:MM:SS` form. Drives the chart x-axis.
3	`PACKET_COUNT`	int	—	Mandatory	Monotonically increasing from 1. Gaps indicate lost packets.
4	`ALTITUDE`	float	m	Mandatory	Barometric altitude. -100 to 15 000. MSL or AGL by firmware convention — state which in the flight software ICD.
5	`PRESSURE`	float	hPa	Mandatory	Static pressure, 10–1100. $\approx$ 1013.25 at sea level.
6	`TEMP`	float	°C	Mandatory	Ambient/board temperature, -40 to $+85$.
7	`VOLTAGE`	float	V	Mandatory	Battery bus voltage, 0–16. Nominal 7.4–8.4 for 2S Li-Po. Colour-coded against a configurable threshold.
8	`NAV_TIME`	string	UTC	Optional	GNSS time as `hhmmss.ss`. May be blank before fix.
9	`LAT`	float	deg	Optional	-90 to $+90$, positive north. Blank or $0, 0$ treated as "no fix".
10	`LON`	float	deg	Optional	-180 to $+180$, positive east.
11	`NAV_ALT`	float	m	Optional	GNSS altitude. Compare against field 4 as a barometer sanity check.
12	`SATS`	int	count	Optional	Satellites in solution, 0–32. Blank becomes 0. UI warns below 6, alerts below 4.
13	`ACC_X`	float	m/s^2	Optional	Body-axis acceleration. ±157 for a ±16 g IMU.
14	`ACC_Y`	float	m/s^2	Optional	As above.
15	`ACC_Z`	float	m/s^2	Optional	As above. $\approx +9.81$ at rest, vertical axis.
16	`GYRO_X`	float	deg/s	Optional	Body-axis angular rate, ±2000.
17	`GYRO_Y`	float	deg/s	Optional	As above.
18	`GYRO_Z`	float	deg/s	Optional	As above. Roll/spin rate about the long axis.
19	`FSM_STATE`	int	enum	Mandatory	Flight state, 0–7. See Table 4. Out-of-range values display as UNKNOWN rather than being rejected.

Field count is fixed at 19

The parser rejects any frame whose body does not split into exactly 19 comma-separated fields. If the flight firmware adds a field, `FIELD_COUNT` in `telemetry_packet.py` and `CSV_HEADER` must both be updated, and every previously recorded CSV becomes a different schema. Version the packet format explicitly if this is ever likely.

2.3 XOR checksum

The checksum is the bitwise exclusive-OR of every byte strictly between the `$` and the `*`, transmitted as two hexadecimal digits:

$$\mathrm{CS} \;=\; b_1 \oplus b_2 \oplus \cdots \oplus b_n, \qquad b_i = \text{ASCII byte } i \text{ of the body.} \tag{2}$$

The implementation is deliberately trivial so that it can be reproduced identically in the Teensy firmware:

```python
def compute_checksum(payload: str) -> int:
    """XOR every byte of *payload* (the text between ``$`` and ``*``)."""
    checksum = 0
    for byte in payload.encode("ascii", errors="replace"):
        checksum ^= byte
    return checksum & 0xFF
```

Listing 1: XOR checksum, from `telemetry_packet.py`.

Worked example

Take the trivial body `ABC`. The three ASCII bytes are XORed in sequence:

Table 3: Worked XOR checksum for the body `ABC`.

Step	Character	Hex	Binary
Start	—	0x00	0000 0000
⊕ byte 1	A	0x41	0100 0001
⊕ byte 2	B	0x42	0100 0010
⊕ byte 3	C	0x43	0100 0011
Result	**CS**	**0x40**	**0100 0000**

Working it through: $0x00 \oplus 0x41 = 0x41$; then $0x41 \oplus 0x42 = 0x03$; then $0x03 \oplus 0x43 = 0x40$. The complete frame is therefore `$ABC*40`.

Note the checksum's properties and limits: XOR detects all single-byte errors and any odd number of bit flips in a given bit position, but it does *not* detect a pair of identical bit flips in the same bit position of two different bytes, nor does it detect transposed bytes. It is adequate for catching the RF bit errors this link produces, and it is what the competition format specifies, but it is not a CRC and should not be described as one.

2.4 Flight state machine

`FSM_STATE` is an integer enumerating the flight computer's state. The dashboard renders it as a large banner filled with a distinct colour per state, so the operator can read the vehicle's state across a field tent at a glance. The colours in Table 4 are the exact values from `FSM_COLORS`.

Every state *transition* is timestamped into the on-screen event log, in the form `FSM ASCENT -> DEPLOY at T+00:00:24`, which gives a post-flight event timeline without needing to reprocess the CSV.

3 Software Architecture

3.1 Module layout

The application is six Python modules with a strict dependency direction: nothing that touches hardware imports anything that touches widgets.

3.1.1 `telemetry_packet.py` — packet model and parser

This is the foundation module and the only one with no dependencies on Qt, pyserial or anything else. That property is deliberate: it means the parser can be exercised by the fuzz harness, by unit tests and by the packet simulator without constructing a `QApplication`, and it means the simulator and the receiver share one definition of the wire format and can therefore never drift apart.

Key contents:

Table 4: Flight state machine states and dashboard colour coding.

Value	Name	Colour	Hex	Meaning
0	BOOT		#6B7785	Power-on self-test; sensors initialising. Altitude not yet valid.
1	TEST_MODE		#00B0D8	Bench/integration mode. Telemetry is live but the vehicle is not flight-armed.
2	LAUNCH_PAD		#E9C135	Armed and on the pad. Ground reference pressure captured; awaiting launch detection.
3	ASCENT		#F07419	Launch detected; powered and coast ascent to apogee.
4	DEPLOY		#E8384F	Apogee detected; recovery deployment event commanded.
5	DESCENT		#8F6BEF	Descending under the primary recovery device.
6	AEROBRAKE_RELEASE		#00B39B	Aerobrake / main recovery released; reduced descent rate.
7	IMPACT		#35C46B	Touchdown detected; vehicle stationary on the ground.

Table 5: Source module responsibilities.

Module	Lines	Responsibility
main.py	∼140	Entry point, CLI arguments, exception hook.
telemetry_packet.py	∼400	Packet dataclass, checksum, parser. No Qt.
serial_worker.py	∼470	Serial thread, ring buffer, frame sync, reconnect.
csv_logger.py	∼300	Logging thread, CSV and error files.
dashboard_ui.py	∼1000	Main window, all widgets and plots.
packet_sim.py	∼520	Synthetic telemetry source and fault injector.

- `TelemetryPacket` — a `@dataclass(slots=True)` holding all 19 fields plus three ground-station annotations: `raw_frame`, `checksum_valid` and `gs_recv_epoch`. Derived properties (`fsm_name`, `fsm_color`, `mission_time_hms`, `has_fix`) keep formatting logic out of the UI.
- `compute_checksum()` and `build_frame()` — the checksum pair, used by both the receiver and the simulator.
- `parse_frame()` — the single entry point that turns one frame string into a `TelemetryPacket` or raises.
- `CSV_HEADER` — the canonical column list, kept adjacent to `to_csv_row()` so the two cannot diverge.
- Exception hierarchy: `PacketError` is the base; `ChecksumError` means the link corrupted the frame; `PacketParseError` means the frame was structurally wrong. A caller that only cares "was this usable" catches the base class.

```
    match = _FRAME_RE.match(text)
    if match is None:
        raise PacketParseError("frame does not match $<body>*<hh>: %r" % text[:120])

    body = match.group("body")
    transmitted = int(match.group("cs"), 16)

    calculated = compute_checksum(body)
    if calculated != transmitted:
        raise ChecksumError(
            "checksum mismatch: got %02X, expected %02X" % (transmitted, calculated)
        )

    fields = body.split(",")
    if len(fields) != FIELD_COUNT:
        raise PacketParseError(
            "expected %d fields, got %d" % (FIELD_COUNT, len(fields))
        )
```

Listing 2: Packet validation flow, from `parse_frame()`.

3.1.2 `serial_worker.py` — ingestion thread

Owns the pyserial handle and everything that happens to bytes before they become packets. Contains three classes:

- `RingBuffer` — a fixed-capacity, drop-oldest byte buffer (8192 bytes) that bounds memory even if the stream is pure garbage.
- `FrameSplitter` — the frame-sync state machine (Section 3.3).
- `SerialWorker(QThread)` — the run loop: connect, read, split, parse, emit, reconnect.

`SerialWorker` exposes its results exclusively through Qt signals, which Qt delivers to the GUI thread through a queued connection automatically:

```
    packet_received = pyqtSignal(object)          # a validated TelemetryPacket
    bad_frame = pyqtSignal(str, str)              # (raw_frame_text, reason)
    stats_updated = pyqtSignal(int, int, int)     # (total, valid, corrupt)
    connection_changed = pyqtSignal(bool, str)    # (is_connected, message)
    log_message = pyqtSignal(str)                 # free-form event log line
```

Listing 3: `SerialWorker` signal interface.

The thread is *long-lived*. It starts once at application launch and stops once at exit. Pressing CONNECT or DISCONNECT does not start or stop the thread; it flips a `_want_connected` flag guarded by a `threading.Lock` that the run loop watches. This is what makes automatic reconnection free: if the USB adapter is unplugged mid-flight, the read raises, the loop closes

the handle and keeps retrying every 1.5 s until the port reappears — with no operator action and no risk of the application dying.

Statistics emission is throttled to 5 Hz. Emitting a signal per packet during a burst would flood the GUI event loop with thousands of queued events, which is precisely the failure mode this architecture exists to prevent.

3.1.3 `csv_logger.py` — disk logging thread

A `QThread` that consumes a `queue.Queue` and writes two files. Disk I/O is kept off both the GUI thread (where a stalled write would freeze the display) and the serial thread (where it would cause the driver's receive buffer to overflow and lose packets).

`queue.Queue` is used rather than a Qt signal because this is a plain producer/consumer handoff with no Qt object affinity involved, and because a *bounded* queue gives explicit back-pressure semantics. If the disk stalls, the oldest rows are dropped and counted, rather than the producer blocking:

```
    def _put(self, item: Tuple[str, Any]) -> None:
        try:
            self._queue.put_nowait(item)
        except queue.Full:
            # Back-pressure: drop the oldest item so live data keeps flowing.
            try:
                self._queue.get_nowait()
                self.rows_dropped += 1
                self._queue.put_nowait(item)
            except (queue.Empty, queue.Full):
                self.rows_dropped += 1
```

Listing 4: Non-blocking, never-raising enqueue, from `csv_logger.py`.

Two behaviours are worth knowing:

- **Lazy file open.** The CSV cannot be named until `TEAM_ID` is known, so the file is opened on the first packet received, not when logging is enabled.
- **Append, never truncate.** `Flight_<TEAM_ID>.csv` is opened in append mode and the header is written only when the file is new or empty. A mid-competition application restart therefore cannot destroy the earlier part of the flight.
- **Flush policy.** The writer flushes at least every 20 rows and at least every 1 second, so a crash costs a fraction of a second of telemetry. A full `os.fsync` is performed only at clean shutdown; calling it at 20 Hz would thrash the disk for no benefit against the realistic failure mode, which is a process crash rather than a power loss on the laptop.

3.1.4 `dashboard_ui.py` — main window

Contains every widget and owns both worker threads. The important classes:

- `ReadoutTile` — a labelled numeric tile with normal / ok / warn / alert colour states.
- `StripChart` — a `pg.PlotWidget` subclass that holds its own data buffers and separates `add_point()` (cheap, per packet) from `redraw()` (timer-driven).
- `GpsTrackPlot` — the lat/lon ground track.
- `Dashboard(QMainWindow)` — layout, signal wiring, timers, connection and logging control, and clean shutdown.

3.1.5 `main.py` — entry point

Sets the High-DPI attributes *before* the `QApplication` is constructed (Qt ignores them afterwards), applies the dark palette, parses `-port`, `-baud`, `-autoconnect` and `-log-dir`, and

installs a `sys.excepthook` that routes an uncaught GUI-thread exception into the on-screen event log and the error file instead of killing the process. A display bug must not end a flight.

3.1.6 `packet_sim.py` — synthetic source

Not part of the flight-day software, but part of the deliverable: it flies a scripted mission and emits real checksum-correct frames so the whole chain can be exercised without a radio. Covered in Section 5.

3.2 Threading model

The application runs exactly three threads, shown in Figure 2.

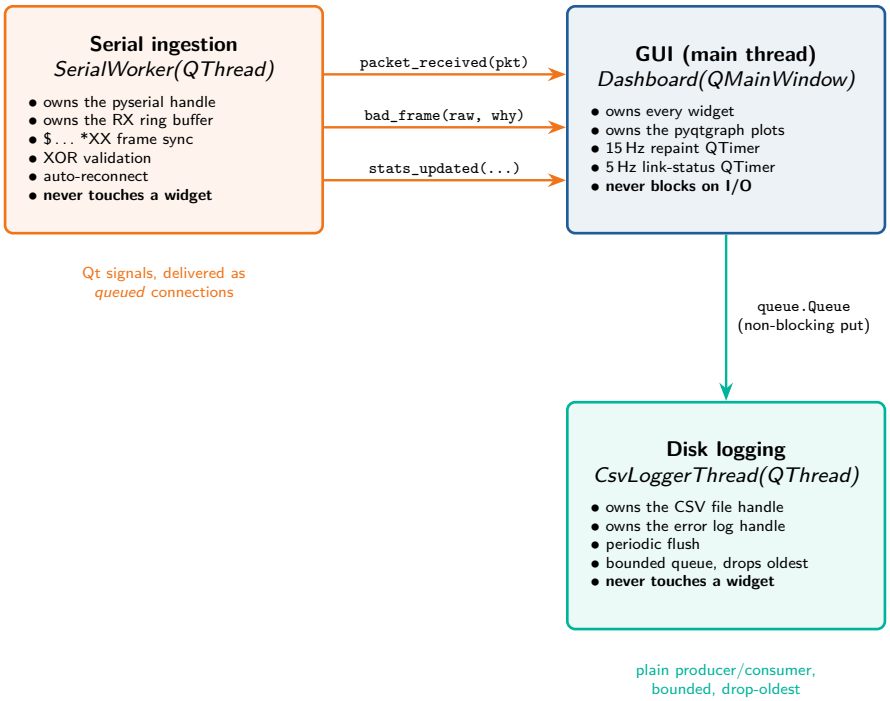


Figure 2: Three-thread model. Ownership is exclusive: exactly one thread may touch the serial port, exactly one may touch the log files, and exactly one may touch widgets.

3.2.1 Why serial I/O is not on the GUI thread

A serial read is a blocking call. Every millisecond the GUI thread spends inside `read()` is a millisecond it is not processing paint events, button clicks or window messages. At 20 Hz with a 50 ms read timeout, a single-threaded design would spend essentially all of its time blocked and Windows would mark the window "Not Responding". Worse, the operator would lose the ability to press DISCONNECT at exactly the moment something had gone wrong.

3.2.2 Why disk I/O is not on the GUI thread

A write to a USB stick, a network drive or a disk that has decided to spin up can block for hundreds of milliseconds. On the GUI thread that is a visible freeze; on the serial thread it is worse, because while the thread is blocked in `write()` it is not draining the driver's receive buffer, so packets are lost on the floor. Moving the writes to a third thread means a slow disk degrades to queued rows, not to lost telemetry.

3.2.3 Render decoupling — the reason the UI never freezes

This is the single most important design decision in the application, and the one most likely to be broken by a well-meaning future edit.

The `on_packet` slot is deliberately cheap. It appends numbers to Python lists, stores the latest packet, and sets a dirty flag. **It repaints nothing.** All drawing happens in a separate `QTimer` callback running at 15 Hz:

```
    def _render(self) -> None:
        """Repaint readouts and charts at a fixed rate, independent of RX rate."""
        try:
            if self._readouts_dirty:
                self._readouts_dirty = False
                self._update_readouts()

                window_s = float(self.window_spin.value())
                autoscale = self.autoscroll_check.isChecked()
                for chart in self.charts:
                    chart.redraw(window_s, autoscale)
                self.gps_plot.redraw()
        except Exception as exc:  # pragma: no cover - defensive
            self.append_event("Render error: %r" % exc)
```

Listing 5: Timer-driven rendering, from `dashboard_ui.py`.

The consequence: whether the radio delivers 5 packets per second or dumps a 500-packet backlog after a dropout, the number of repaints per second is constant at 15. Repainting six pyqtgraph plots is the expensive operation in this application; decoupling it from arrival rate is what satisfies the "tolerate bursts without freezing" requirement.

> **Do not repaint from `on_packet`**
>
> If a future change calls `setData()`, `setText()` or any other widget method directly from the `packet_received` slot, the application will appear to work perfectly at 10 Hz on the bench and will lock solid the first time the link recovers from a dropout and delivers a burst. Add data to the buffers; let the timer draw.

3.3 Ring buffer and frame synchronisation

3.3.1 The problem

The serial stream is not frame-aligned. A single `read()` may return half a packet, two and a half packets, or a packet with 40 bytes of RF noise in front of it. The receiver must find packet boundaries in an arbitrary byte stream, and must do so in a way that cannot be permanently derailed by any input.

3.3.2 The ring buffer

Incoming bytes are appended to a `RingBuffer` of fixed 8192-byte capacity — roughly 64 full frames, or over 3 seconds of backlog at 20 Hz, far more than a healthy reader ever accumulates. When the buffer is full the *oldest* bytes are evicted and counted in `dropped_bytes`. This guarantees bounded memory: a stream of pure garbage, or a transmitter stuck sending a continuous carrier, cannot grow the process without limit.

It is implemented over a `bytearray` with front-trimming rather than a wrapped index pair, because the frame scanner needs *contiguous* memory to run `bytes.find()` over. The externally visible behaviour — fixed capacity, drop-oldest — is what matters.

3.3.3 The frame-sync rules

`FrameSplitter._extract()` repeatedly applies three rules until no more complete frames can be produced:

1. **Discard leading junk.** Everything before the first `$` is not part of any frame. Drop it. This is what silently absorbs RF noise, partial frames received before the receiver started, and — as discussed in Section 6.1.4 — XBee API frame headers.
2. **Restart on a second `$`.** If another `$` appears before the terminating `*`, the first frame was truncated in flight. Abandon it and restart parsing at the newer `$`. Every such event increments a `resyncs` counter.
3. **Bound the frame length.** If the buffer grows past `MAX_FRAME_LEN` (512 bytes) without producing a terminator, the leading `$` is spurious. Drop that one byte and rescan. This rule is what prevents a lost `*` from wedging the reader forever.

If none of the rules fire and the terminator simply has not arrived yet, the scanner returns and waits for more bytes — the partial frame stays in the buffer across calls, which is what makes byte-at-a-time reassembly work.

```
        while True:
            start = buf.find(b"$")
            if start < 0:
                # No frame start at all: the whole buffer is junk.
                buf.clear()
                break
            if start > 0:
                buf.consume(start)  # rule 1: drop leading junk

            star = buf.find(b"*", 1)
            next_dollar = buf.find(b"$", 1)

            if 0 <= next_dollar and (star < 0 or next_dollar < star):
                # rule 2: truncated frame, restart at the newer '$'
                buf.consume(next_dollar)
                self.resyncs += 1
                continue

            if star < 0:
                # Terminator has not arrived yet.
                if len(buf) > MAX_FRAME_LEN:
                    buf.consume(1)  # rule 3
                    self.resyncs += 1
                    continue
                break
```

Listing 6: Frame-sync loop, from `serial_worker.py`.

3.3.4 Worked recovery example

Consider this hostile stream, which is exactly what the fault injector produces:

`qk3$%z9 $TEAM,...,3*4A \r\n $TEAM,12.5,88, $TEAM,...,4*7C`

1. `qk3` precedes the first `$` — discarded by rule 1. (The `$` inside the junk becomes the candidate start; the following `%z9` then fails rule 2 or the checksum, and is discarded.)
2. The first complete frame is found, checksum verified, emitted as a packet.
3. `\r\n` precedes the next `$` — discarded by rule 1.
4. The truncated frame has no `*` before the next `$` — abandoned by rule 2, `resyncs` incremented. No packet is emitted and nothing is corrupted.
5. The final frame parses normally.

Net result: two valid packets recovered, one resync, zero exceptions, zero false packets. This exact behaviour is verified by test **GCS-T-04** in Section 5.

3.3.5 Three layers of defence

The robustness requirement is met by three independent layers, so that a failure of one does not reach the operator:

1. **Structural** — the frame-sync rules discard anything that does not look like a frame, without ever raising.
2. **Cryptographic (weakly)** — the XOR checksum rejects frames whose bytes were altered in flight. These are counted as *corrupt* and shown in the link status panel.
3. **Defensive** — `_handle_frame()` wraps `parse_frame()` in `try/except` that catches `ChecksumError`, `PacketParseError`, the `PacketError` base class *and* bare `Exception`. A latent bug in the parser itself therefore costs one packet, not the ingestion thread:

```
        except Exception as exc:
            # A bug in the parser must not kill the ingestion thread mid-flight.
            self.corrupt_packets += 1
            self.bad_frame.emit(frame, "unexpected %s: %s" % (type(exc).__name__, exc))
            return
```

Listing 7: Catch-all in the ingestion path, from `serial_worker.py`.

Every rejected frame, from any of the three layers, is forwarded to the error log with its raw bytes and the reason — unconditionally, even when CSV logging is switched off, so that a post-flight link investigation always has data.

4 Dashboard UI Reference

4.1 Overall layout

The window is organised as a connection bar across the top, a fixed-width status column on the left, and a 3×2 grid of strip charts filling the remainder. The theme is dark by design: the station is read outdoors and in a field tent, where a white background is both glare-prone and a significant battery drain on an LCD backlight.

4.2 Connection bar

Table 6: Connection bar controls.

Control	Function
Port	Editable dropdown of detected serial devices. Also accepts a typed pyserial URL such as `socket://127.0.0.1:5555`. See Section 6.2.
RESCAN	Re-enumerates serial ports without restarting the application. Use after plugging in the XBee adapter.
Baud	Editable baud selector, default 9600. Must match the radio's `BD` setting — see Section 6.3.
CONNECT	Opens the link. Turns red and reads DISCONNECT while connected.
START LOGGING	Toggles CSV writing. The error log is always written regardless of this setting.
CSV: ...	Full path of the active flight log, shown once the first packet has arrived and the file name is known.
CONNECTED	Link indicator: green when the port is open, red when it is not.

4.3 Flight state banner and live telemetry

The banner fills with the state colour from Table 4 and shows the state name and numeric value, e.g. DESCENT [5]. It is pinned above the scrolling region so it can never be scrolled out of view on a short laptop screen.



Figure 3: Main dashboard during a simulated flight. Left column: flight-state banner, live telemetry tiles, GPS/NavIC panel with ground track, link status and display settings. Right: altitude, pressure, temperature, battery voltage, accelerometer and gyroscope strip charts.

Below it, eight tiles show Team ID, Packet Count, Mission Time (`HH:MM:SS`), Altitude, Pressure, Temperature, Battery and Satellites. Two tiles are colour-coded:

- **Battery** — green at or above the configurable threshold (default 7.00 V), amber below it, red below 90 % of it. The threshold is set in the Display Settings panel and should be set to the flight computer's brown-out margin, not to an arbitrary round number.
- **Satellites** — green at 6 or more, amber at 4–5, red below 4. Below 4 satellites there is no 3D position solution at all.

A field showing `-` means the flight computer transmitted a blank or unparseable value for an *optional* field; the rest of the packet is still valid and still logged.

4.4 Strip charts

Six pyqtgraph plots share a common x-axis definition: mission time in seconds, taken from the packet `TIMESTAMP` field, falling back to ground-station elapsed time if the flight computer sends a timestamp that cannot be interpreted.

The visible time window is set by Chart window (s) in Display Settings, default 60 s, range 5–1800 s. Charts are pannable and zoomable with the mouse; unticking Auto-scale Y axes freezes the vertical range so a manually chosen scale is not overridden.

4.5 GPS / NavIC panel

The four readouts show `LAT`, `LON`, `NAV_ALT` and `NAV_TIME`. Latitude and longitude turn amber when the packet does not carry a usable fix — either the fields were blank, or they contained the $0, 0$ null-island default that many receivers emit before lock.

The ground track plots longitude on x against latitude on y: a faint blue path of every fix received in the session, with a red crosshair marking the most recent one. Note that this is a

Table 7: Strip chart contents.

Chart	Series	What to look for
Altitude	1 (blue)	The flight profile. Should be monotonic up to apogee, then monotonic down. Steps or spikes indicate barometer noise or a pressure-port problem.
Pressure	1 (orange)	The mirror image of altitude. If altitude moves and pressure does not, the altitude is being computed from a stale reading.
Temperature	1 (red)	Should fall with altitude at roughly 6.5 °C per 1000 m. A rise during ascent suggests self-heating or a sensor near the electronics.
Battery voltage	1 (green)	Slow droop is normal. A sharp sag correlated with a deployment event indicates the recovery charge is loading the same bus as the avionics.
Accelerometer	3 (X/Y/Z)	$\approx +9.81\,\mathrm{m/s^2}$ on the vertical axis at rest. The launch transient and the deployment shock should both be clearly visible.
Gyroscope	3 (X/Y/Z)	Roll rate about the long axis. A high, non-decaying spin rate under the recovery device indicates a tangled or asymmetric deployment.

[SCREENSHOT PLACEHOLDER]

Save the image as `docs/figures/gps_panel.png`

It will be included automatically on the next compile.

Figure 4: GPS/NavIC panel: latitude, longitude, navigation altitude and navigation time readouts above the two-dimensional ground track.

plain Cartesian scatter, not a georeferenced map — see Section 8. For recovery purposes, read the numeric latitude and longitude, not the plot.

4.6 Link status panel

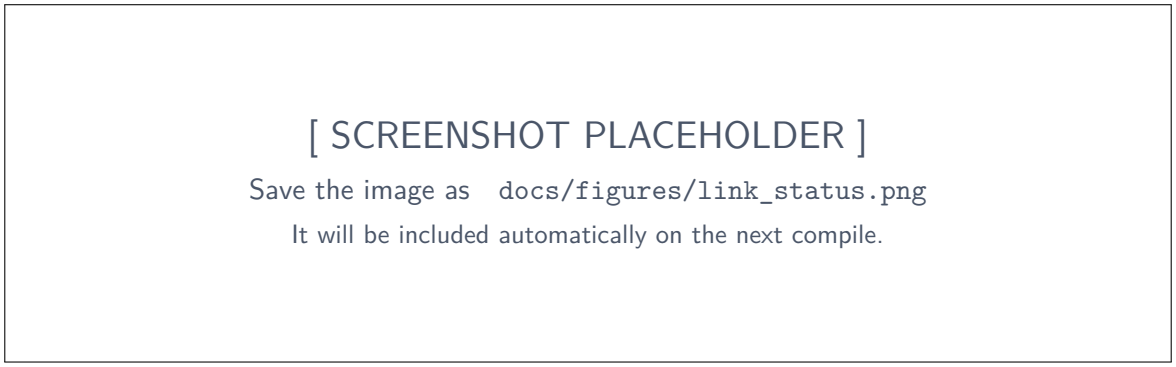


Figure 5: Link status panel showing packet rate, last packet age, valid/total packet counts, corrupt packet count and the link health banner.

This panel is the operator's primary health indicator for the RF link.

Table 8: Link status fields.

Field	Meaning and interpretation
Packet rate	Valid packets per second, averaged over a trailing 3-second window. Should sit near the firmware's transmit rate. A rate well below it means packets are being lost on the RF link or the serial line is oversubscribed (Section 6.3).
Last packet age	Seconds since the last *valid* packet. Green under 2 s, red above.
Valid / total pkts	Valid packets over total frames recovered from the stream. The difference is the number rejected. A widening gap is the earliest warning of a degrading link.
Corrupt packets	Frames that were structurally complete but failed checksum or parsing. Turns red as soon as it is non-zero. A handful over a flight is normal; a steadily climbing count means the link is marginal.
Health banner	AWAITING TELEMETRY before the first packet, LINK NOMINAL in green while healthy, !! TELEMETRY STALE !! in red after 2 s without a valid packet.

Every stale/recovered transition is also written to the event log and the error log with a timestamp, giving a dropout record for post-flight analysis.

4.7 Display settings and event log

Chart window (s), Battery warning (V) and Auto-scale Y axes are described above. Clear plots & counters wipes the on-screen history and zeroes the counters; it does *not* touch the CSV, which continues uninterrupted.

The event log at the bottom of the column carries timestamped entries for connections, disconnections, port errors, FSM transitions, stale/recovered events and logging start/stop. The most recent entry is mirrored in the status bar at the bottom of the window.

5 Testing and Validation Performed

5.1 Approach

Verification was performed at three levels: pure-logic tests against the parser and framer with no Qt involved; integration tests running the real `Dashboard` with real worker threads against

a real socket, driven offscreen; and a windowed visual check. All tests ran against the delivered source, on Python 3.11 with PyQt5 5.15.11, pyqtgraph 0.14.0 and pyserial 3.5.

5.2 Test matrix

Table 9: Ground station verification test matrix.

Test ID	Method	Result observed	Verdict
GCS-T-01	Build a frame with `build_frame()`, parse it back with `parse_frame()`; compare all 19 fields.	All fields recovered exactly; `ASCENT` resolved from state 3; mission time rendered `00:00:12`; CSV row length equals `CSV_HEADER` length.	**PASS**
GCS-T-02	Corrupt the two checksum digits of a valid frame and parse.	`ChecksumError` raised; frame rejected, not displayed.	**PASS**
GCS-T-03	Parser fuzz: 4013 malformed inputs — empty strings, bare delimiters, 600-char frames, doubled frames, truncated frames, `NaN` fields, and random byte strings drawn from an alphabet including `NUL` and `0xFF`.	Zero unexpected exceptions. Every input either parsed or raised a `PacketError` subclass. No crash, no hang.	**PASS**
GCS-T-04	Frame sync: a stream of leading garbage, one good frame, one truncated frame with no terminator, a second good frame and a third with no trailing `CRLF`, fed **one byte at a time**.	All 3 good frames recovered and parsed; 1 resync recorded on the truncated frame; no false packets emitted.	**PASS**
GCS-T-05	Ring buffer bound: feed 200 000 bytes of non-frame data.	Buffer held at 8192 bytes; 283 747 bytes counted as dropped across the test; memory bounded.	**PASS**
GCS-T-06	Wedge resistance: feed `$` followed by 100 000 bytes with no terminator, then a valid frame.	Rule 3 forced resync; the subsequent valid frame was still recovered. Reader did not wedge.	**PASS**
GCS-T-07	Simulator coverage: sample the mission profile across the full timeline and parse every frame.	All frames valid; all 8 FSM states $\{0 \ldots 7\}$ produced, confirming full state coverage for UI testing.	**PASS**
GCS-T-08	Fault-injected stream: 3000 frames at 15 % corrupt, 15 % garbage-prefixed, 15 % truncated, 10 % dropped, through the real framer and parser.	1909 valid packets recovered, 362 checksum failures correctly detected, 0 malformed escapes, 425 resyncs. No unexpected exception.	**PASS**

continued on next page

Table 9 continued from previous page

Test ID	Method	Result observed	Verdict
GCS-T-09	Live integration: real `Dashboard`, real `SerialWorker` and `CsvLoggerThread`, real TCP socket, simulator at 20 Hz in `-chaos` mode, 16-second run.	259 valid packets, 16 corrupt correctly rejected, 275 total frames. Readouts, all six charts and the ground track updated live.	**PASS**
GCS-T-10	GUI responsiveness under burst: independent 50 ms watchdog timer during GCS-T-09, including a 40-packet burst after 3 s of silence.	Watchdog fired continuously throughout; event loop never blocked; no "Not Responding" state.	**PASS**
GCS-T-11	Stale-link detection: observe the banner across the simulator's 3-second silence gap.	Banner switched to TELEMETRY STALE after 2 s, logged the transition, and returned to LINK NOMINAL on recovery, logging that too.	**PASS**
GCS-T-12	Auto-reconnect, four transitions: (a) connect with no peer present; (b) start the peer; (c) kill the peer mid-stream; (d) restart the peer.	(a) retried without crashing, connected=False; (b) auto-connected, 94 packets; (c) loss detected, connected=False; (d) auto-recovered to 212 packets. No operator action at any step.	**PASS**
GCS-T-13	Clean shutdown and CSV integrity: close the window during an active logging session, then inspect the file.	Both threads stopped and joined; file flushed and `fsync`ed. 263 lines = 1 header + 262 rows, matching the 262 packets received. Header matched `CSV_HEADER` exactly.	**PASS**
GCS-T-14	Logger back-pressure: monitor `rows_dropped` during the 20 Hz chaos run.	0 rows dropped, 262 written, 20 error-log entries. Queue never saturated.	**PASS**
GCS-T-15	Error archival: verify rejected frames reach `errors.log` with raw bytes and reason.	Each entry carried an ISO-8601 UTC timestamp, the failure reason including expected vs. received checksum, and the complete raw frame.	**PASS**

5.3 Fault injection with `packet_sim.py`

`packet_sim.py` flies a scripted mission — BOOT through IMPACT in approximately 75 seconds — and emits genuine checksum-correct frames. It imports `build_frame()` from `telemetry_packet.py`, so the simulator and the receiver are guaranteed to agree on the wire format.

By default it runs a TCP server rather than writing to a COM port. The reasoning is given in Section A.2.

```
# Terminal 1: hostile link
python packet_sim.py --rate 20 --chaos
```

```
3
4  # Terminal 2: the ground station
5  python main.py
6  #   Port -> "socket://127.0.0.1:5555  (packet_sim.py)"  -> CONNECT -> START LOGGING
```

Listing 8: Running the fault-injection scenario.

Table 10: Fault injection modes and the real-world failure each simulates.

Flag	-chaos	Real-world condition simulated
-corrupt-rate	0.06	A bit error in flight. One payload byte is altered while the original checksum is kept, so the frame is structurally perfect and *only* the XOR check can catch it. Exercises the checksum layer specifically.
-garbage-rate	0.03	RF noise, a partially received frame, or another transmitter on channel. Random non-frame bytes are injected ahead of a frame. Exercises frame-sync rule 1.
-truncate-rate	0.03	The link dropping mid-transmission, or the receiver's buffer overflowing. A frame is cut off part-way. Exercises frame-sync rule 2.
-drop-rate	0.05	A packet lost entirely — fading, antenna null, or momentary obstruction. Exercises packet-count gap visibility and the rate calculation.
-burst	40	Recovery from a dropout: the link goes silent for 3 s, then delivers a 40-packet backlog at once. Exercises the stale banner *and* the render decoupling described in Section 3.2.

Each rate is independently settable, so a specific layer can be exercised in isolation — for example `-corrupt-rate 1.0` with all other rates zero proves that no corrupt frame ever reaches the display.

5.4 Recommended pre-flight regression

Before every launch campaign, run the following and confirm the results against Table 9:

1. `python packet_sim.py -rate 20 -chaos -seed 42` against a live dashboard for at least 90 seconds (a full simulated mission).
2. Confirm the FSM banner passes through all eight states.
3. Confirm Corrupt packets is non-zero (proving the checksum layer is active) while all displayed values remain physically plausible (proving no corrupt frame leaked through).
4. Close the window and verify the CSV row count matches the valid packet count shown in the link status panel.

6 Connecting to Real Hardware

This section is the operational core of the document. Sections 6.1 through 6.7 take the station from an unconfigured radio to a verified flight log.

6.1 Ground-side XBee setup

6.1.1 Hardware required

- One Digi XBee 3 PRO (XB3-24AST), physically identical to the flight unit. Label it GROUND with a marker at the outset — once both radios are configured they are visually indistinguishable, and swapping them mid-campaign is a genuinely difficult fault to diagnose.
- One XBee USB adapter (SparkFun XBee Explorer USB, Digi XBIB-U, or equivalent) providing a USB-to-UART bridge and the 2 mm-pitch XBee socket.

- One USB A-to-mini/micro cable as required by the adapter.
- The ground antenna and, if the connectors differ, an SMA/RP-SMA adapter (Section 6.6).

6.1.2 Wiring

The XBee Explorer handles all of the wiring internally; there is nothing to solder. Two points still matter:

- **Orientation.** The XBee module is keyed by its flat-topped outline, which must match the silkscreen on the adapter. Inserting it reversed applies 3.3 V to the wrong pins and will destroy the module.
- **Never power the radio without an antenna.** Transmitting into an open connector reflects the full output power back into the PA. Attach the antenna *before* applying USB power, every time. On the ground side the radio transmits only 802.15.4 acknowledgements, but the rule is absolute and it is easier to make it a habit than an exception.

6.1.3 Configuration in XCTU

Install Digi XCTU, connect the Explorer, and use Discover radio modules. Once the module appears, set the parameters in Table 11. **Every one of these must be identical on the flight radio and the ground radio**, with the sole exception of the destination/source address pair.

Table 11: XCTU parameters for the ground radio (CanSat values shown).

Parameter	Value	Notes
`CH`	`17` (hex)	Channel 23 = 2465 MHz for CanSat. Use `0B` (channel 11 = 2405 MHz) for the rocket. Must match the flight unit exactly.
`ID`	e.g. `2465`	PAN ID. Any 16-bit value, but it must match the flight unit and should be unique among teams operating simultaneously.
`AP`	`0` or `1`	Serial API mode. See Section 6.1.4 — read it before choosing.
`BD`	`7` (115200)	Serial baud. **Do not leave this at the 9600 default** — see Section 6.3.
`MM`	`0`	Standard 802.15.4 MAC mode with acknowledgements.
`CE`	`0`	End device. Set the flight unit or the ground unit as coordinator per your network plan; only one may be.
`DH`/`DL`	flight unit's `SH`/`SL`	Destination address. Read the flight radio's serial number high/low and enter them here, and vice versa, for a strict point-to-point pair.
`PL`	`4`	Power level. Maximum for range; reduce for close-range bench testing to avoid receiver desensitisation. Confirm the maximum permitted level against your module's datasheet and your IN-SPACe authorisation.

After changing any parameter, click Write in XCTU so the settings are committed to the module's non-volatile memory. A parameter that is only *queued* in the XCTU pane is lost at the next power cycle — a classic cause of "it worked yesterday".

Record the final configuration of both radios in the team logbook. When the link fails in the field, the first diagnostic step is comparing the radios against that record.

6.1.4 API mode versus transparent mode

This deserves a precise statement, because the GCS behaves differently from what the packet specification might suggest.

The GCS synchronises on ASCII, not on 0x7E API frames

This software does *not* contain an 802.15.4 API frame decoder. It scans the serial byte stream for the ASCII sequence `$ ... *XX` directly. It has no knowledge of frame type `0x90`, of 64-bit source addresses, or of the API length and checksum fields.

The practical consequences:

Transparent mode (`AP = 0`) — recommended for bring-up.
The radio delivers the payload bytes verbatim. The serial stream is exactly the ASCII telemetry the flight computer wrote, and the frame sync is operating on precisely what it was designed for. This is the simplest and most predictable configuration, and it is what should be used for the first link test in Section 6.4.

API mode (`AP = 1`) — works, with a caveat.
The stream contains `0x7E`-delimited binary frames with the ASCII payload embedded inside each `0x90` receive frame. Because frame-sync rule 1 discards everything before a `$`, the binary API header is silently skipped as inter-frame junk and the payload is extracted correctly. This works, and it was the behaviour assumed by the original packet specification. The caveat is that a binary address or length byte can coincidentally equal `0x24` (`$`) or `0x2A` (`*`), producing an occasional false frame start. Such a frame fails the XOR checksum and is counted as corrupt — it never reaches the display or the CSV — but it will inflate the corrupt-packet counter, which makes that counter a less reliable indicator of true RF quality.

Recommendation: use `AP = 0` on the *ground* radio unless the team specifically needs API features such as per-packet RSSI or source addressing. If API mode is required, expect a non-zero corrupt count as a baseline, characterise it on the bench, and treat departures from that baseline — not the absolute value — as the link-quality signal. Adding a native API frame decoder is listed in Section 8.

6.2 Selecting the correct COM port

6.2.1 Identifying the port

The XBee Explorer enumerates as a USB-to-serial bridge (FTDI FT231X on SparkFun boards, Silicon Labs CP2102 on many clones). Windows assigns it a `COMn` number.

The reliable identification method, which takes ten seconds and removes all ambiguity:

1. With the adapter *unplugged*, press RESCAN in the dashboard and note the ports listed.
2. Plug the adapter in, wait for the enumeration chime, press RESCAN again.
3. The port that appeared is the XBee. Its description will read something like `COM7 -- USB Serial Port (FTDI)`.

The equivalent from a terminal, useful when the dashboard is not yet running:

```
python -c "import serial.tools.list_ports as p; [print(x.device, '|', x.description, '|', x.hwid) for x in p.comports()]"
```

Listing 9: Enumerating serial ports from the command line.

The `hwid` field carries the USB vendor and product ID — for example `VID:PID=0403:6015` for an FTDI FT231X — which identifies the adapter model unambiguously even when several serial devices are present.

6.2.2 Selecting it in the dashboard

The Port dropdown is populated from the detected devices plus one synthetic entry, `socket://127.0.0.1:5555 (packet_sim.py)`, which exists for simulator testing. For real

hardware, simply select the real `COMn` entry instead — the simulator entry needs no removal and has no effect when not selected.

The dropdown is editable, so a device name or a pyserial URL can also be typed directly. The port can also be pre-selected from the command line:

```
python main.py --port COM7 --baud 115200
python main.py --port COM7 --baud 115200 --autoconnect
```

Listing 10: Pre-selecting a port at launch.

6.2.3 When several COM ports appear

Field laptops routinely show three or four. Work through these in order:

1. **Use the plug/unplug differential** described above. This is definitive and should be the default method.
2. **Check Device Manager** (Ports (COM & LPT)). The adapter appears by chipset name. Its properties dialog gives the VID/PID for cross-checking.
3. **Eliminate the known impostors.** Bluetooth serial ports (`Standard Serial over Bluetooth link`) often occupy several low COM numbers and will open successfully but deliver no data — producing the confusing symptom of a green CONNECTED indicator with a TELEMETRY STALE banner. The Teensy 4.1's own USB serial port will also appear if it is plugged into the same laptop; do not select it, as it carries the flight computer's debug output, not the radio downlink.
4. **Just try one.** Selecting the wrong port is harmless. If no packets arrive within a few seconds, press DISCONNECT and try the next. Nothing is written to the wrong device.
5. **Pin the assignment** once identified. In Device Manager, Port Settings → Advanced → COM Port Number lets you fix the number so it survives re-plugging into a different USB socket. Do this before the launch campaign and write the number on the adapter with tape.

6.3 Baud rate matching and the serial budget

The baud rate selected in the dashboard must equal the `BD` parameter written to the ground radio. There is no negotiation and no error message: a mismatch produces a stream of framing errors that the parser sees as unrecoverable garbage. The symptom is characteristic — CONNECTED in green, packet rate zero, corrupt count either zero (nothing resembles a frame) or climbing rapidly (occasional accidental `$`).

The 9600 default cannot carry 20 Hz telemetry

A 125-byte frame plus `CRLF` is 127 bytes. At 20 Hz that is 2540 bytes/s, or 25 400 bit/s at the usual 10 bits per byte (1 start + 8 data + 1 stop). A 9600 baud line carries 960 bytes/s and therefore tops out at **7.6 packets per second**. Running the mission rate over a 9600 baud link will silently truncate and drop packets in the radio's UART buffer, before this software ever sees them. Set `BD` = 7 (115200) on *both* radios and select 115200 in the dashboard.

Headroom matters because the frame length is not constant: a negative coordinate, an extra digit of altitude or a longer team ID all lengthen it. Size the link for the worst case, not the nominal.

Note that this is the *serial* budget between the radio and the laptop. The over-the-air budget is separate: 802.15.4 at 2.4 GHz has a 250 kbit/s raw rate, so 25.4 kbit/s of payload is comfortable, but real throughput falls with range and retries. If the packet rate drops as the vehicle gets further away while the serial line is provably fast enough, the limit is RF, not UART.

Table 12: Serial throughput budget for a 127-byte frame (125-byte frame plus CRLF), at 10 bits per transmitted byte.

BD	Baud	Bytes/s	Max packets/s	Verdict for 20 Hz
3	9600	960	7.6	**Inadequate** — default; do not use
4	19200	1 920	15.1	**Inadequate**
5	38400	3 840	30.2	Marginal — 1.5× headroom
6	57600	5 760	45.4	Adequate — 2.3× headroom
7	115200	11 520	90.7	**Recommended** — 4.5× headroom

6.4 First real link test

Perform this on the bench, with the flight radio powered but *not* yet connected to the Teensy. The objective is to prove the RF link and the ground software in isolation, before any flight-firmware variable is introduced. Keep the two radios at least 2–3 m apart: at close range with PL at maximum, the receiver can be saturated and the link will paradoxically perform worse than at distance.

Step 1 — Configure both radios. Set CH, ID, BD and the DH/DL address pair per Table 11, and AP = 0 on both for this test. Write the settings to both modules.

Step 2 — Start the ground station. Launch the GCS, select the ground adapter's COM port, set the baud rate to match BD, and press CONNECT. The indicator turns green and the banner reads AWAITING TELEMETRY. Leave START LOGGING off for now.

Step 3 — Prepare a known-good packet. Generate one valid frame with the project's own checksum routine, so that any mismatch is attributable to the link and not to hand arithmetic:

```
# Run from the project directory:  python make_test_frame.py
from telemetry_packet import build_frame

print(build_frame(
    "2025ASI001,1.00,1,320.00,975.00,25.00,8.20,000000.00,"
    "32.726600,74.857000,320.00,7,0.00,0.00,9.81,0.00,0.00,0.00,1"))
```

Listing 11: make_test_frame.py — generates one valid test frame.

This prints a complete frame with a correct checksum, for example $2025ASI001,1.00,1,320.00,...,1*XX. Copy it exactly.

Step 4 — Transmit it from the flight radio. Put the flight radio on a second USB adapter, open XCTU's Console (or any terminal at the matching baud rate), and send the frame followed by Enter. In transparent mode every byte typed is transmitted.

Step 5 — Verify reception. Within one screen refresh the dashboard should show:

- Team ID 2025ASI001 and Packet Count 1;
- Altitude 320.0 m, Pressure 975.00 hPa, Temperature 25.0 °C, Battery 8.20 V in green;
- Satellites 7, latitude 32.726600, longitude 74.857000, and a single point on the ground track;
- the flight state banner in cyan reading TEST_MODE [1];
- Valid / total pkts showing 1/1 and Corrupt packets showing 0;
- the banner going to TELEMETRY STALE about two seconds later, which is correct — only one packet was sent.

Step 6 — Deliberately break it. This step is not optional; it proves the validation layer is armed rather than merely quiet. Send the same frame with one character of the *body* altered but the original checksum retained. The dashboard must show Corrupt packets increment to `1` while the readouts remain unchanged, and `logs/errors.log` must contain the raw frame with a `checksum mismatch` reason. If the corrupt count stays at zero, the checksum is not being enforced and the problem must be found before flying.

Step 7 — Sustained-rate check. Send frames repeatedly at the mission rate (XCTU's console supports repeat transmission, or connect the simulator to the flight-side adapter with `python packet_sim.py -serial COM11 -baud 115200 -rate 20`). Confirm the packet rate settles near 20 pkt/s and the corrupt count stays low. If the rate plateaus near 7.6 pkt/s, the link is running at 9600 baud — return to Section 6.3.

Step 8 — Log and verify. Enable START LOGGING, capture a minute of traffic, close the window, and confirm `logs/Flight_2025ASI001.csv` exists with a correct header and the expected number of rows.

6.5 After Teensy 4.1 integration

6.5.1 What changes on the ground: nothing

This is worth stating plainly, because it is the point of the interface. When the flight radio moves from a bench USB adapter to the Teensy 4.1's UART, *no ground-station change is required*:

- the same COM port — the ground adapter has not moved;
- the same baud rate — provided the Teensy's `Serial1.begin()` matches the radio's `BD`;
- the same parser — the wire format is unchanged;
- the same CSV schema and file name.

The only new failure mode is on the flight side: the Teensy–XBee UART. Check that the Teensy TX pin goes to the XBee `DIN` pin (not TX-to-TX), that grounds are common, and that the XBee is powered from a clean 3.3 V rail. The XBee 3 PRO's transmit current draw is substantial and a marginal regulator will brown out the radio precisely when it transmits — which presents at the ground station as packets that stop the moment the vehicle powers up its other subsystems.

6.5.2 What to verify

1. **Packet rate stabilises near 20 Hz.** It should be steady, not oscillating. A rate that wanders indicates the firmware's transmit loop is not time-deterministic — check for blocking sensor reads or `delay()` calls in the main loop.
2. **Packet count increments by exactly one per packet.** Gaps mean loss. Compare the gap count against the corrupt count: gaps with a low corrupt count point to RF dropouts, whereas gaps with a high corrupt count point to a marginal link or a serial overrun.
3. **FSM state reflects reality.** On the bench the state should read `BOOT` then `TEST_MODE`, and must *not* advance to `LAUNCH_PAD` or beyond. If it does, the launch-detection threshold is too sensitive and will trigger on handling.
4. **Mission time advances at 1 s per second.** If it advances faster or slower, the firmware's time base is wrong and every time-derived quantity in the flight log will be wrong with it.
5. **Sensor values are physically plausible.** Pressure near the local station value, temperature near ambient, Z acceleration near $+9.81\,\mathrm{m/s^2}$ with the vehicle upright, gyroscope near zero at rest. This is the last opportunity to catch a swapped axis or a units error before flight.

6. **Battery voltage matches a multimeter reading.** If the telemetered voltage is off by more than a few tens of millivolts, the ADC divider calibration is wrong — and the battery warning threshold is therefore meaningless.
7. **GPS acquires a fix outdoors.** Satellite count should climb above 6 and the ground track should show a tight cluster while stationary. A wandering track while stationary indicates poor antenna placement or multipath.
8. **Corrupt count stays low over 10 minutes.** Establish the baseline now, in a known-good configuration, so that a departure from it in the field is meaningful.

6.6 Field deployment

6.6.1 Ground antenna

The ground station uses a 7-element cross-Yagi, connected to the XBee's coaxial interface.

- **Check the connector gender and polarity first.** SMA and RP-SMA are mechanically compatible but electrically incompatible: the centre pin and socket are swapped. Connecting RP-SMA to SMA gives a joint that mates perfectly and passes almost no RF. If the link is dead with everything else correct, this is a prime suspect. Carry the correct adapter and test the assembly on the bench before the campaign.
- **Attach the antenna before applying power.** As in Section 6.1: no power without a load on the connector.
- **Keep the pigtail short.** Coaxial loss at 2.4 GHz is significant — thin RG-174 pigtails can lose on the order of 1 dB per metre. Every decibel lost in the feedline is a decibel of link margin gone. Mount the radio close to the antenna and run USB (which is loss-tolerant) rather than a long coaxial run.
- **Pointing.** A 7-element Yagi has a main lobe of roughly 40–50° and meaningful gain only within it. The cross-Yagi (two orthogonal Yagis fed in quadrature) gives circular polarisation, which removes the polarisation loss that occurs when a tumbling vehicle's linearly-polarised antenna rotates relative to the ground station — this is exactly why it is used here. Point it at the vehicle, not at the sky in general, and track it through the flight. Assign one team member solely to antenna pointing; it is not a task to combine with operating the dashboard.
- **Mind the nulls.** A Yagi has very low gain off the back and directly off the ends of its elements. During a high, near-vertical flight the vehicle can pass through the antenna's null even at short range. A brief TELEMETRY STALE at apogee with a directional antenna is often geometry, not a fault.

6.6.2 Laptop power for a session over two hours

- **Assume no mains power.** Bring a fully charged laptop plus a USB-C PD power bank of 20 000 mAh or more if the laptop supports USB-C charging, or a small inverter and a sealed lead-acid battery if it does not. Test the whole arrangement before the campaign.
- **Disable USB selective suspend.** This matters more than it sounds. Windows will power down an idle USB serial adapter to save battery, which drops the COM port. The GCS will auto-reconnect (Section 3.2), so the failure is survivable, but each event costs seconds of telemetry. Set the power plan to High performance, then Change advanced power settings → USB settings → USB selective suspend setting → Disabled, on battery as well as plugged in.
- **Disable sleep, hibernate and the lid-close action.** A laptop that sleeps during the flight loses the flight.
- **Disable automatic updates and restarts** for the duration of the campaign.

- **Reduce screen brightness rather than allowing dimming timeouts.** The dark theme already helps here.
- **Bring a sunshade.** An LCD in direct sunlight is unreadable at any brightness, and a laptop in direct sun in the field will thermally throttle. A cardboard hood over the screen is adequate and costs nothing.
- **Bring a spare configured XBee, a spare USB adapter and a spare cable.** These are the cheapest items in the system and the most likely to fail.

6.6.3 RF troubleshooting: the stale-telemetry checklist

When !! TELEMETRY STALE !! appears during an actual flight, work through Table 13 in order. It is ordered by *probability* × *speed of check*, not by severity — the aim is to restore the link fastest, and during flight there are seconds, not minutes.

Table 13: Stale telemetry troubleshooting sequence.

#	Check	How to test	Indication
1	Antenna pointing	Sweep the Yagi towards the vehicle's last known bearing.	Rate recovers as it comes into the main lobe. Most common in-flight cause.
2	Connection indicator	Look at the top-right of the window.	Red DISCONNECTED means the *USB* link dropped, not the RF link. The station auto-reconnects; check the cable.
3	Corrupt count behaviour	Watch whether corrupt packets are still climbing.	Climbing = signal present but marginal (range/obstruction). Frozen = no signal at all (pointing, PAN/channel, or the vehicle transmitter).
4	Obstruction	Note terrain, buildings, vehicles, and people between the antenna and the vehicle.	2.4 GHz does not penetrate well. A person standing in the beam is enough. Raise the antenna and clear the line of sight.
5	Range and geometry	Compare the last GPS fix with the antenna position.	Loss at apogee on a near-vertical flight is often the antenna null, not range. Re-point rather than assuming a failure.
6	Cable and connector	Wiggle the SMA joint and the USB cable gently.	Intermittent recovery localises the fault immediately.
7	Flight battery	Look at the last logged voltage before the dropout.	A voltage falling towards the threshold before the dropout means the flight unit is browning out — an on-board fault, not an RF one.
8	Channel / PAN ID	Compare both radios against the logbook record.	Only relevant if the link never worked in this session. A link that worked and then stopped has not changed channel.

continued on next page

Table 13 continued from previous page

#	Check	How to test	Indication
9	Baud rate	Confirm the selector matches `BD`.	As above: only a suspect if the link never worked. Rate plateauing near 7.6 pkt/s indicates 9600 baud.
10	Interference	Note Wi-Fi access points, other teams' radios, Bluetooth.	2465 MHz overlaps Wi-Fi channel 11–13. Ask nearby teams what channel they are on.

Do not restart the application to "fix" a stale link

The station recovers from serial disconnection automatically and keeps the CSV open across the event. Restarting loses the in-memory plot history and costs several seconds of telemetry during the part of the flight where it is least replaceable. The CSV is appended, not truncated, so a restart will not destroy data — but there is no reason to accept the gap. Work the checklist instead.

Note that a dropout costs only the packets sent during the outage. The station keeps running, the flight state banner holds the last known state, and reception resumes automatically the moment the link returns.

6.7 CSV handoff for competition judging

6.7.1 Where the files are

Logs are written to a `logs/` subdirectory of the working directory from which `main.py` was launched, or to the path given by `-log-dir`. Two files are produced:

Table 14: Output files.

File	Contents
`logs/Flight_<TEAM_ID>.csv`	Every valid packet. 26 columns: the 19 telemetry fields, plus ground-station receive time in both ISO-8601 UTC and epoch form, plus normalised mission time in seconds and `HH:MM:SS`, plus the `checksum_valid` flag, the resolved FSM state name, and the complete raw frame.
`logs/errors.log`	Every rejected frame with an ISO-8601 UTC timestamp, the rejection reason, and the raw bytes. Also carries session notes: connects, disconnects, stale and recovery events.

`TEAM_ID` is taken from the first packet received and sanitised for use as a file name. If the flight computer transmits `2025ASI001`, the file is `Flight_2025ASI001.csv`.

6.7.2 Retention of the raw frame

Every row carries the original frame text in the `raw_frame` column. This is deliberate and is worth defending to a judging panel: it means the CSV is not merely the ground station's *interpretation* of the telemetry, it contains the received data itself. Any dispute about a parsed value can be settled by recomputing the checksum and re-parsing the raw frame from the log, independently of this software.

6.7.3 Pre-handoff verification

Run this checklist before submitting. It takes two minutes.

1. **Close the application cleanly** with the window close button. This flushes and `fsync`s the file. Do not kill the process, and do not copy the CSV while the application is still running.
2. **Confirm the file name** matches the team ID exactly as registered with the competition.
3. **Verify the header row.** It must be present, exactly once, as the first line:

```
Get-Content logs\Flight_2025ASI001.csv -TotalCount 1
```

   Compare against `CSV_HEADER` in `telemetry_packet.py`.
4. **Check the row count** against the Valid / total pkts figure from the link status panel. They must agree, allowing for the header line:

```
(Get-Content logs\Flight_2025ASI001.csv | Measure-Object -Line).Lines
```

5. **Confirm `checksum_valid` is 1 throughout.** Only validated packets are written, so any 0 indicates a defect and should be investigated before submission.
6. **Sanity-check the flight profile.** Open the CSV and confirm the altitude column rises and falls, mission time increases monotonically, and `fsm_state` progresses through the expected sequence without impossible jumps.
7. **Check for an appended prior session.** Because the file is opened in append mode, a restart adds rows to the existing file. If the packet count column restarts partway through, the file contains more than one session — split it or annotate it before handing it over.
8. **Archive both files together**, plus `errors.log`. The error log demonstrates that link impairments were detected and quantified rather than ignored, which is a point in the team's favour at review, not against it.
9. **Make two copies** on separate physical media before leaving the field.

7 Live Map and KML Export

Two features share one source of truth. Every position fix of a session is recorded once, in `track_store.py`, from `Dashboard.on_packet`; the map draws from it and the KML export writes from it. That is deliberate: the track an operator watched has to be the track a recovery team later flies from, and two independent copies would eventually disagree.

A fix is recorded only when `has_fix` is true, latitude and longitude are finite and in range, and the point is not exactly $0, 0$ — a receiver without a lock transmits zeros, and a flickering lock transmits small plausible-looking values near Null Island. Intake is a single append, so a 20 Hz stream costs nothing measurable on the GUI thread, and fixes are recorded whether or not the map window is open.

7.1 Live map

MAP, next to **CSV TABLE**, opens a non-modal window drawing 256 px Web Mercator (EPSG:3857) tiles with `QPainter`. QtWebEngine was rejected: it is a browser engine, a second process and roughly 100 MB of dependency for what is a tiled image and a few polylines.

7.1.1 Offline is the base case

The launch site has no internet, so `.mbtiles` files in `assets/maps/` are read directly with the standard library's `sqlite3` and are selected automatically whenever a pack exists. Online sources are the convenience, not the other way round.

MBTiles stores rows in TMS order, whose origin is the *bottom* left, while slippy-map y counts from the top. The conversion is $y_{\text{TMS}} = 2^z - 1 - y$ and it is applied in exactly one place. Getting it wrong produces a map that is vertically mirrored but otherwise entirely plausible, which is

the kind of fault that survives a casual look; a unit test asserts it in both directions against a generated file.

A tile that cannot be had — no pack, no network, nothing cached — is drawn as a grey square labelled with its zoom level. A hole in the map must read as a hole rather than as the vehicle having flown over blank ground.

7.1.2 Threading

The GUI thread never blocks on a tile. Network tiles go through `QNetworkAccessManager`, which is asynchronous on the GUI thread, with at most four requests in flight and a `User-Agent` naming this application as providers' usage policies require. MBTiles reads run on a `QThreadPool` as `QRunnable`s, one `sqlite3` connection per job, because sqlite objects cannot be shared across threads. Decoded tiles land in a bounded LRU so a long session cannot grow memory without limit. `paintEvent` cannot raise: it counts failures and keeps the frame, because Qt will simply call it again.

Repaints are coalesced to 5 Hz. Measured with the simulator at 20 Hz, the GUI thread's tick distribution with the map open was indistinguishable from the same session with it closed.

7.1.3 Display

Vehicles are drawn by `payload_type` with a distinct colour *and* shape — a circle for the CanSat, a triangle for the rocket — so the two remain distinguishable in a monochrome screenshot or to a colour-blind operator. Each carries a label with GNSS altitude and the age of its last fix, which turns red after five seconds. The full track is kept; only the drawing is thinned, and never at the endpoints. An optional ground station position adds slant range and true bearing to each vehicle and is stored in `map_settings.json`; with no file and no coordinates entered, nothing is drawn and nothing is reported.

7.1.4 Sources

Sources are declared in `map_sources.json`, which is gitignored because it may name an API key's environment variable and local paths; the tracked template is `map_sources.example.json`. A missing or unreadable file is not an error — the built-in list is used and one line goes to the event log.

Source	Status
Offline (MBTiles)	Any pack in `assets/maps/`. Default when present.
ISRO Bhuvan	WMS at `bhuvan-vec1.nrsc.gov.in`, verified answering anonymously with EPSG:3857 tiles. A *vector* server: administrative boundaries, not satellite imagery.
OpenStreetMap	Light interactive use only; bulk download refused.
Mappls / MapmyIndia	Shipped disabled. Needs a key, and their terms have not been reviewed for this project.

Table 15: Map sources. No Google Maps or Google tile server is used anywhere.

Any online source necessarily learns which tiles are being viewed, which is to say the approximate location and zoom of the launch site. Where that matters, build a pack in advance and fly offline.

7.1.5 Building a pack

`tools/fetch_tiles.py` writes a valid MBTiles file, rate limited to two requests per second, resumable (an already-stored tile is skipped, so an interrupted run continues), and refusing jobs over 20,000 tiles without `--force`. A 5×5 km site at zoom 12–18 is 2,218 tiles.

It refuses any source not explicitly marked `"bulk_download": true` and prints why. Bulk downloading is a sustained load on someone else's server and is governed by most providers' terms; the OpenStreetMap Foundation's Tile Usage Policy forbids it outright for `tile.openstreetmap.org`. QGIS (*Generate XYZ tiles (MBTiles)*) and Mobile Atlas Creator are documented alternatives in the README.

7.2 KML export

EXPORT KML, next to **START LOGGING**, writes the session's full tracks — not the thinned drawing data — through a save dialog, defaulting to the `logs/` folder and the CSV's own `Flight_<team>_<timestamp>` naming. It works while logging is running. With no fixes recorded it says so and writes nothing, rather than leaving an empty file that would look like a failed flight.

`kml_export.build_kml` is pure: tracks and options in, a string out, with no Qt, no clock and no file I/O, so the whole document is testable without a display or a radio. Everything is serialised through `xml.etree.ElementTree`, so a team ID or vehicle name containing `&` or `<` is escaped by the serialiser rather than by hand.

Each vehicle gets a Folder containing a `gx:Track` whose paired `<when>` and `<gx:coord>` elements drive Google Earth's time slider, with `ExtendedData` arrays for FSM state, satellite count and packet count; an extruded `LineString` of the same path, visible with the slider parked; and Launch, Apogee and Last-known placemarks carrying a telemetry table each.

Coordinate order. KML coordinates are `lon,lat,alt` — the reverse of the usual order — and `gx:coord` separates them with spaces while `<coordinates>` uses commas. Both are asserted by tests, because either mistake produces a file that parses cleanly and plots in the wrong place.

Altitude is `nav_alt_m`, the GNSS height above mean sea level, with `altitudeMode absolute`. The barometric `altitude_m` is deliberately not used: it is referenced to a fixed 1013.25 hPa sea level with no ground zero-set, so it carries a constant offset that would place the entire track at the wrong height. A link transmitting no GNSS altitude — the current raw-CSV firmware, for instance — produces a `clampToGround` path, and the document description says so rather than drawing the flight at a height nobody measured.

Before writing, no-lock and exact $0, 0$ fixes are dropped, duplicate packet counts are dropped, and single-sample GPS jumps are dropped (default: more than 500 m in under one second, which is 1800 km/h). All three counts are written into the document description, so a shortened path is never silent.

`tools/csv_to_kml.py` converts a flight CSV after the event through the same builder and the same filters, reading columns by `CSV_HEADER` name rather than position, so a post-flight export is identical to a live one.

8 Known Limitations and Future Work

The following are known, deliberate gaps. They are recorded here so that a review panel sees them acknowledged rather than discovered, and so the team can prioritise them.

8.1 Functional limitations

No command uplink.
The station is receive-only. There is no way to arm, disarm, calibrate, trigger deployment or reset the flight computer from the ground. This is currently a scope decision rather than an oversight — see Section 1.2 — but a competition requiring ground-commanded events would need a transmit path, an uplink packet format, and an acknowledgement scheme.

No native 802.15.4 API frame decoder.
As detailed in Section 6.1.4, the receiver synchronises on ASCII rather than decoding `0x7E` API frames. API mode works because the binary header is discarded as inter-frame junk, but per-packet RSSI, source addressing and delivery status are not available, and occasional false frame starts inflate the corrupt counter. A proper `0x90` frame decoder in front of the existing ASCII parser would fix all of these and would let the station display received signal strength per packet — which would be genuinely valuable for antenna pointing.

Ground track is not a map.
The GPS panel is a plain Cartesian scatter of longitude against latitude. There is no map tile background, no coastline, no scale bar and no distance-to-landing readout. Degrees of longitude are also not plotted to the same physical scale as degrees of latitude, so the track's *shape* is distorted — at the reference latitude used here, one degree of longitude is about 84 % of the ground distance of one degree of latitude. For recovery, use the numeric coordinates. Adding an offline tile cache, a local ENU projection and a range-and-bearing-from-launch-site readout is the single highest-value improvement available.

No RSSI or link margin display.
Signal strength is not shown, because it is not in the packet and is not available without the API decoder above. Link health is inferred indirectly from packet rate and corrupt count. Adding an RSSI field to the telemetry packet from the flight side would be the simpler of the two routes.

No packet-loss statistic.
The station counts corrupt frames but does not currently compute lost packets from gaps in `PACKET_COUNT`. This is straightforward to add and would give a direct link-quality percentage rather than an inferred one.

No session replay.
A recorded CSV cannot be played back through the dashboard. Post-flight analysis requires external tools. A replay source reading a CSV and feeding the existing parser would be a small addition and would make post-flight debriefs considerably easier.

Single-vehicle only.
One serial port, one packet stream, one team ID. Simultaneous CanSat and rocket telemetry would require two instances of the application, which works but does not correlate the two streams.

8.2 Technical debt and platform notes

Python version constraint.
PyQt5 has no binary wheels for Python 3.13 or later. The virtual environment must be built against Python 3.10–3.12. A future migration to PyQt6 would lift this, at the cost of a moderate API update across `dashboard_ui.py`.

Plot history is bounded.
Each series holds at most 40 000 points — about 33 minutes at 20 Hz. Beyond that the oldest points are dropped from the *display*. The CSV is unaffected and remains complete.

Mission-time x-axis assumes monotonicity.
The strip charts locate the visible window with a binary search over the mission-time array. A flight computer that resets its time base mid-flight produces a non-monotonic axis and a briefly

incorrect window. It cannot crash the application, but the display will be misleading until the buffer scrolls past the discontinuity.

Append-mode CSV can merge sessions.
As noted in Section 6.7, restarting the application appends to the existing file rather than creating a new one. This is the safe default — it cannot destroy data — but it puts the burden of detecting merged sessions on the handoff check. Timestamped file names would remove the ambiguity at the cost of breaking the required naming convention.

No automated test suite in the repository.
The verification in Section 5 was executed against the delivered code, but the harnesses were run as one-off scripts rather than committed as a `pytest` suite. Committing them would let the pre-flight regression in Section 5.4 run as a single command and would catch regressions introduced by future edits.

8.3 Suggested priority order

If the team has limited time before the next design review, the highest value per hour of work is, in order:

1. Commit the verification harnesses as a `pytest` suite (low effort, protects everything else).
2. Add packet-loss computation from `PACKET_COUNT` gaps (low effort, directly improves link diagnosis).
3. Add CSV replay (low effort, large benefit to post-flight debriefs).
4. Add range and bearing from the launch site to the GPS panel (moderate effort, directly serves recovery).
5. Add the `0x90` API frame decoder with RSSI display (moderate effort, improves antenna pointing and link diagnosis).
6. Add an offline map tile background (highest effort, mostly presentational).

A Quick Reference

A.1 Installation and launch

```
# PyQt5 requires Python 3.10-3.12; 3.13+ has no wheels.
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

# Launch
.\.venv\Scripts\python.exe main.py

# Launch pre-configured for the real radio
.\.venv\Scripts\python.exe main.py --port COM7 --baud 115200 --autoconnect
```

Listing 12: Environment setup and launch (Windows PowerShell).

On Linux or macOS: `python3.11 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt && python main.py`.

A.2 Simulator

Windows provides no built-in mechanism for creating a virtual COM port pair. The usual solution, the `com0com` driver, involves unsigned-driver installation that is impractical on a locked-down competition laptop.

pyserial resolves this: `serial.serial_for_url("socket://host:port")` presents a TCP connection through the same `Serial` API. Because the GCS opens its port through

`serial_for_url`, pointing it at `socket://127.0.0.1:5555` exercises the *entire* real code path — ring buffer, frame sync, checksum, threading and logging — with no driver installed. That entry is pre-loaded in the port dropdown.

```
python packet_sim.py                      # TCP server, 10 Hz, clean link
python packet_sim.py --rate 20            # competition nominal rate
python packet_sim.py --rate 20 --chaos    # full fault injection
python packet_sim.py --serial COM11       # write to a real/virtual COM port
python packet_sim.py --stdout             # print frames to the console
python packet_sim.py --help               # all options
```

Listing 13: Simulator invocations.

A.3 Configuration constants

Table 16: Tunable constants and where they live.

Constant	Module	Default	Effect
RING_CAPACITY	serial_worker.py	8192	RX ring buffer size in bytes.
MAX_FRAME_LEN	telemetry_packet.py	512	Longest accepted frame; forces resync above this.
READ_TIMEOUT_S	serial_worker.py	0.05	Serial read timeout; bounds shutdown latency.
RECONNECT_DELAY_S	serial_worker.py	1.5	Interval between reconnection attempts.
RENDER_HZ	dashboard_ui.py	15	Plot and readout repaint rate.
STATUS_HZ	dashboard_ui.py	5	Link-status refresh rate.
STALE_AFTER_S	dashboard_ui.py	2.0	Staleness threshold in seconds.
RATE_WINDOW_S	dashboard_ui.py	3.0	Averaging window for packets/second.
MAX_PLOT_POINTS	dashboard_ui.py	40000	Per-series display history cap.
QUEUE_MAX	csv_logger.py	1000	Logger queue depth before dropping oldest.
FLUSH_INTERVAL_S	csv_logger.py	1.0	Maximum time between flushes.
FLUSH_EVERY_ROWS	csv_logger.py	20	Maximum rows between flushes.

A.4 Symptom index

End of document — Ground Control Station Technical Documentation, Revision 1.0

Table 17: Common symptoms and their usual causes.

Symptom	Usual cause
Green CONNECTED, but AWAITING TELEMETRY forever	Wrong COM port (often a Bluetooth port), or `CH`/`ID` mismatch between radios. Section 6.2.
Green CONNECTED, corrupt count climbing fast, valid count zero	Baud rate mismatch. Section 6.3.
Packet rate plateaus near 7.6 pkt/s	Link running at 9600 baud; it physically cannot carry 20 Hz. Section 6.3.
Rate is correct but corrupt count grows steadily	Marginal RF: range, obstruction, antenna pointing, or receiver saturation at very close range.
DISCONNECTED appears spontaneously	USB dropout, frequently USB selective suspend. The station auto-reconnects. Section 6.6.
Latitude and longitude amber, ground track empty	No GPS fix yet, or the receiver is emitting the 0, 0 null-island default. Check satellite count.
Some fields display –	Optional fields blank or unparseable in the packet. The packet is still valid and still logged.
Battery tile red on the bench	Threshold set above the actual bus voltage, or the flight-side ADC divider is uncalibrated. Section 6.5.
CSV missing after a session	Logging was never enabled, or no packet ever arrived (the file is created lazily on the first packet).
CSV contains two flights	Append mode plus an application restart. Section 6.7.